Cloud Computing Assignment Report

Name: Sumit | Roll number: 230101024 | GitHub: https://github.com/drop04/cloud-project | Release: https://github.com/drop04/cloud-project/releases/tag/models-v1 (tag `models-v1`)

Overview and architecture

A 3.8B teacher (phi3.5 via Ollama) labels 3,000 commands; the cleaned replies are the only training targets for a 70M student (pythia-70m), which is stored in object storage and served by a FastAPI service on a local k3d cluster.

Component	Where	Role
Ollama + phi3.5	host laptop	teacher, temperature 0, seed 42
SeaweedFS	k3d, namespace `data`	S3 storage, buckets `datasets` and `models`
Teacher workers	namespace `pipeline`	label commands in parallel shards
Training	host CPU, Python 3.11	fine-tunes pythia-70m on cleaned teacher replies
Parser service	namespace `app`	Deployment (2 replicas), Service, Ingress (Traefik), HPA 1 to 2
GitHub Actions	GitHub	lint, unit tests, evaluation from the Release, quality gate, secret scan

Key decisions. Sequence-level distillation (the student is trained on the teacher's text replies). Model weights live in object storage and are downloaded at start-up, never baked into the image or committed. The service reports ready only after the model loads, so Kubernetes sends no traffic to a loading Pod. Rolling updates use `maxSurge: 1, maxUnavailable: 0`.

1. Task 1: Object storage and parallel teacher labelling

Setup and decisions

- Cluster created from `cluster/k3d.yaml`; SeaweedFS runs in namespace `data`; buckets `datasets` and `models` exist; `commands_pool.jsonl` is in `datasets`; prompt is in `prompts/teacher_prompt.txt`.
- Teacher: phi3.5 served by Ollama on the host (temperature 0, seed 42). Workers in namespace `pipeline` reach it at `host.k3d.internal:11434`.

Sharding. A command at line number i (counting from 0) belongs to the worker whose `SHARD_INDEX` equals i mod `SHARD_COUNT`. With `SHARD_COUNT` = 2, each of the 2 workers handles 1,500 commands and writes its own object, `datasets/teacher/shard-N.jsonl`, so workers never write to the same object and need no coordination. A worker flushes its progress to its shard object every 25 records and, when restarted, continues from the lines already there instead of starting over. `clean.py` merges all shards and keeps records in `id` order, so the result does not depend on which worker finished first, which is what makes the checksum reproducible.

Measured results

Workers and run time (`kubectl -n pipeline get pods`, `kubectl -n pipeline get job teacher-labelling`):

```
kubectl -n pipeline get pods :
NAME                         READY   STATUS      RESTARTS   AGE
curltest                     0/1     Completed   0          26h
teacher-labelling-0-2zjgv    0/1     Completed   0          26h
teacher-labelling-1-vw5l4    0/1     Completed   0          26h

kubectl -n pipeline get job teacher-labelling :
NAME               STATUS     COMPLETIONS   DURATION   AGE
teacher-labelling  Complete   2/2           4h59m      26h
```

`python scripts/check_shards.py` (every id in exactly one shard):

```
teacher/shard-0.jsonl 1500
teacher/shard-1.jsonl 1500
pool 3000 seen 3000 gaps 0 repeats 0
```

Counts from `python worker/clean.py`:

Input	(a) not valid JSON	(b) wrong keys	(c) bad intent	(d) slot not in command	Duplicates	Kept
3,000	1,551	85	2	34	16	1,312

Check: 3,000 = 1,551 + 85 + 2 + 34 + 16 + 1,312.

Checksum: first run `9818935679bba46c6e099216c81ab2a859a8b09d1ff7738034440972f7bf6620`; second run `9818935679bba46c6e099216c81ab2a859a8b09d1ff7738034440972f7bf6620`; identical: yes; matches

`datasets/v1/train.sha256`: yes (the printed digest is the one written to that file, and sha256sum of data/train.jsonl gives the same value).

Assumptions and interpretation

Rule (a) removed 1,551 of 3,000 teacher replies (51.7%) because they were not a single valid JSON object. On the validation set, 66 of 100 raw teacher replies failed strict parsing; the three examples inspected were a correct JSON object followed by a "Note:" paragraph. The two rates differ (51.7% against 66%), and what share of the 1,551 failures are trailing notes was not checked. Only 1,312 records survived cleaning, so the student trained on all of them (the 1,500 limit in training/config.yaml was not reached). Labelling took 4 h 59 min on this laptop and would differ on other hardware.

2. Task 2: Training the student (sequence-level distillation)

Setup and decisions

Training examples have the form `Command: <command>` then `JSON: <teacher reply>`, using all 1,312 records of the cleaned dataset (the limit in the config is 1,500). Everything is read from `training/config.yaml`.

Setting	Value
Student	EleutherAI/pythia-70m (70M parameters, English, Apache 2.0)
Seed	42
Learning rate	5e-5
Batch size	8
Epochs	3
Max sequence length	128
Libraries	python 3.11, torch 2.5.1, transformers 4.46.3, numpy 1.26.4, pyyaml 6.0.2
v2 differs by	seed 7 (v1: 42) and 4 epochs (v1: 3); output in models/v2 and training/loss_v2.csv

Suitability of pythia-70m. It is a small base model with no instruction tuning, so it cannot follow the task zero-shot (base JSON validity 0%, below). With 1,312 training examples it reached 99% JSON validity, but intent accuracy is only 70% (date 79%, time 87%, place 86%).

Measured results

Training loss per epoch (`training/loss.csv`):

Epoch	v1 loss	v2 loss
1	0.1990	0.2032
2	0.0778	0.0779
3	0.0590	0.0605
4	not run (v1 has 3 epochs)	0.0561

Artefacts: `models/v1/` and `models/v2/` in bucket `models`: the cold-start log shows the v1/ files downloaded from the bucket (section 4.5) and v2 answered 302 requests during the rolling update (section 4.3). Checksum of the local tarball: `d27538f6539a040fbd80a89b34b6518004fe3724f41c5975b9726f6361830aed models/model-v1.tar.gz`, matches `models/v1/model.sha256`: yes.

Scores on `validation.jsonl` (100 commands), all in percent. Files: `results/{base,teacher,student}_validation.json`.

System	JSON validity	Intent	Date	Time	Place	Full frame
Base pythia-70m	0.0	0.0	0.0	0.0	0.0	0.0
Teacher phi3.5 (strict, official)	30.0	26.0	23.0	27.0	29.0	17.0
Student v1	99.0	70.0	79.0	87.0	86.0	41.0
Supplementary: teacher, first line only	89.0	72.0	68.0	78.0	76.0	40.0

The supplementary row applies the same cut at the first newline that base and student outputs already get; the strict row is the official one. Teacher outputs scored invalid under strict parsing: 66 of 100 (for example a correct JSON object followed by "Note: The term 'upcoming' is used to indicate a future date..." and by "Note: The time phrase has been simplified to match the format given...").

What changed (measured)

Training raised validity from 0% to 99% and full-frame accuracy from 0% to 41%. Intent (70%) is the weakest score and caps full-frame accuracy. Student full-frame accuracy (41) is about equal to the first-

line teacher (40) and above the strict teacher (17). Final training loss was 0.0590 for v1 (3 epochs, seed 42) and 0.0561 for v2 (4 epochs, seed 7).

Assumptions and interpretation

With only 100 validation examples, a 1 to 2 point gap is within noise, so the student is described as matching the teacher, not beating it.

Distillation explained

(a) Sequence-level distillation trains the student on the teacher's generated output sequences (here, the JSON text) using the ordinary next-token loss, as if they were gold labels.

(b) Logit-based distillation trains the student to match the teacher's per-token output probability distribution, typically with a KL-divergence loss, so the student also learns how likely the teacher considered the alternatives.

(c) Why it is not possible here. The teacher is queried through Ollama and the worker keeps only the generated text, so the teacher's per-token distributions are not recorded; and phi3.5 and pythia-70m use different tokenizers and vocabularies, so their per-token distributions could not be matched position by position anyway.

(d) A real limitation. The student trained on only 1,312 of the 3,000 labelled commands (44%), because cleaning discarded the rest. Cleaning checks format and that slot values appear in the command, not whether the intent is right, so the student also learns the teacher's wrong labels; the teacher itself reaches only 40% full-frame (first-line scoring).

3. Task 3: The inference service in a container

Setup and decisions

FastAPI service in `service/` on port 8000: `POST /parse` (200 with the four-key JSON plus `X-Model-Version` and `X-Served-By`), `GET /healthz` (200 while alive), `GET /readyz` (503 until the model is loaded, then 200). Configuration comes from `MODEL_URI`, `MODEL_VERSION`, `S3_ENDPOINT`, `MAX_NEW_TOKENS`; credentials come only from a Kubernetes Secret. Every output is validated against `schema.json`; invalid output returns 500 `{"error":"invalid_model_output"}`. `FAULT_INJECT=1` replaces the model output with `not json` before validation. Image: `python:3.11.11-slim`, CPU-only `torch==2.5.1+cpu`, all dependencies pinned, no weights (downloaded from object storage at start-up).

Measured results

Image size (`docker images parser-service:local`, limit 2 GB):

```
IMAGE                 ID             DISK USAGE   CONTENT SIZE   EXTRA
parser-service:local   d7576d52c38a       1.19GB             0B
```

No weights in the image: a find for *.safetensors and pytorch_model.bin inside the image returns 0 files.

Endpoints and headers (`/readyz` while loading and after: first block below; `/parse` with headers):

```
14:20:02 /readyz -> 000
14:20:02 /readyz -> 000
14:20:02 /readyz -> 000
14:20:03 /readyz -> 503
14:20:03 /readyz -> 503
   ... (16 polls in total returned 503)
14:20:06 /readyz -> 503
14:20:06 /readyz -> 200
HTTP/1.1 200 OK
date: Fri, 06 Nov 2026 08:50:06 GMT
server: uvicorn
content-length: 14
content-type: application/json
```

The container was not listening for the first three polls (000), `/readyz` returned 503 for about 3 s while the model loaded, then 200. Output of a `/parse` request, with headers:

```
HTTP/1.1 200 OK
Content-Length: 67
Content-Type: application/json
Date: Thu, 05 Nov 2026 14:52:57 GMT
Server: uvicorn
X-Model-Version: v1
X-Served-By: parser-78b786b767-fp8t9

{"intent":"weather_query","date":null,"time":null,"place":"london"}
```

50 sequential requests at `--cpus 1 --memory 1g` (`results/task3_memory.txt`):

```
limits: --cpus 1 --memory 1g
requests ok: 50 / 50
OOMKilled: false
peak memory (max of docker stats samples): 342.9 MiB
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O    PIDS
1c1ef00dec1c   memtest   0.13%     341.9MiB / 1GiB     33.39%    286MB / 670kB   0B / 270MB   18
```

Fault injection (`results/fault_inject_500.txt`):

```
HTTP/1.1 500 Internal Server Error
date: Thu, 05 Nov 2026 21:14:05 GMT
server: uvicorn
content-length: 32
content-type: application/json

{"error":"invalid_model_output"}
```

Assumptions and interpretation

Peak memory is the highest of samples taken about every 0.5 s with docker stats (each call adds some delay), so a shorter spike could be missed.

4. Task 4: Kubernetes deployment, scaling and reliability

Setup and decisions

Item	Setting
Objects	Deployment, Service, Ingress (`/parse` to Service port 80), HPA, all in namespace `app`
Replicas	start at 2
Requests / limits	CPU 500m / 1, memory 512Mi / 1Gi
Probes	liveness `/healthz`; readiness `/readyz`
Update strategy	RollingUpdate, `maxSurge: 1`, `maxUnavailable: 0`
HPA	min 1, max 2, target CPU 70%

Limitation. The cluster is one node on one laptop, so it shows Pod-level recovery only; it does not demonstrate fault tolerance across machines, and a Pod restart is not a machine failure. The HPA reacts with a delay: replicas rose 16 s after CPU first read 111% of the target, and scale-down waits the default 5 minutes, so capacity lags behind changes in load.

Measured results

4.1 No ready endpoints while the model loads (`results/endpoints_loading.txt`):

```
NAME     ENDPOINTS         AGE
parser   10.42.0.18:8000   20h
parser   <none>            20h
parser                     20h
parser   10.42.0.20:8000   20h
```

4.2 HPA scaling (`results/hpa_timeline.txt`, selected rows: time, then NAME REFERENCE TARGETS MINPODS MAXPODS REPLICAS AGE):

```
20:10:11 parser   Deployment/parser   cpu: 0%/70%   1     2     1     24m
20:10:19 parser   Deployment/parser   cpu: 111%/70%   1     2     1     24m
20:10:35 parser   Deployment/parser   cpu: 111%/70%   1     2     2     24m
20:10:51 parser   Deployment/parser   cpu: 199%/70%   1     2     2     24m
20:16:21 parser   Deployment/parser   cpu: 122%/70%   1     2     2     30m
20:16:37 parser   Deployment/parser   cpu: 0%/70%   1     2     2     30m
20:17:49 parser   Deployment/parser   cpu: 0%/70%   1     2     2     31m
```

Replicas rose from 1 to 2 at 20:10:35, 16 s after CPU first read 111% of the 70% target. The capture ended at 20:17:49 with CPU at 0% and replicas still 2, because the HPA's default 5-minute scale-down delay had not yet elapsed. A second capture (results/hpa_scaledown.txt, no timestamps), taken with the Deployment set to 2 replicas and no load, shows REPLICAS going from 2 to 1:

```
NAME     REFERENCE           TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
parser   Deployment/parser   cpu: 0%/70%   1         2         2          17h
parser   Deployment/parser   cpu: 0%/70%   1         2         2          17h
parser   Deployment/parser   cpu: 0%/70%   1         2         2          17h
parser   Deployment/parser   cpu: 0%/70%   1         2         1          17h
```

4.3 Rolling update v1 to v2, then rollback (`results/rolling_stats.csv`, `rolling_headers.json`, `after_rollback.txt`):

Requests	Failures	Failure %	Version split during test	`X-Model-Version` after rollback
1,334	29	2.17%	v1: 1,006 (76.9%), v2: 302 (23.1%)	v1

4.4 Recovery (`results/recovery_before.txt`, `recovery_replicas.txt`, `recovery_stats.csv`): replicas before 2 of 2 Ready, after deleting a Pod 1 of 2 Ready, back to 2 of 2 Ready by the next 5 s sample; requests 793, failures 8 (1.01%).

4.5 Cold start (`results/coldstart_times.txt`, `coldstart_logs.txt`): Pod creation to Ready, three runs with two Pods each: 5.0 s and 5.0 s, 6.0 s and 6.0 s, 5.0 s and 5.0 s; median 5.0 s over all six Pods. Log evidence of the download from object storage:

```
downloading v1/config.json (799 bytes)
downloading v1/model.safetensors (281715176 bytes)
downloading v1/model.sha256 (82 bytes)
```

4.6 Load balancing (two Ready replicas, standard load test through the Ingress, `results/loadtest_headers.json`):

Replica	Successful requests	Share
parser-78b786b767-h2rrw	257	50.6%
parser-78b786b767-rbbqb	251	49.4%

Summary against targets

Test	Measured	Target	Met
Rolling update failures	2.17% (29 of 1,334)	at most 0.5%	No
Version after rollback	v1	v1	Yes
Recovery failures	1.01% (8 of 793)	at most 2%	Yes
Replica count restored	2 of 2 Ready by the next 5 s sample	yes	Yes
Balance, smaller share	49.4%	at least 30% each	Yes
HPA	1 to 2 at 20:10:35 under load; 2 to 1 in a second capture with no load	1, then 2, then 1	Yes

Assumptions and interpretation

The three runs without a rollout (recovery, HPA, standard load test) show 0.98% to 1.01% failures, about one per 100 requests. The load test cycles through the 100 validation commands and the student returns invalid JSON for one of them (validity 99%), so the service answers that command with HTTP 500 on every cycle. In the rolling-update run, results/rolling_failures.csv lists 15 HTTP 500 responses (1.12% of 1,334, the same baseline) and 14 HTTP 502 responses (1.05%) between 14:48:33 and 14:49:50. The 502s alone are above the 0.5% target. The load generator and the cluster also share one laptop's CPU.

5. Task 5: Subnetting and network access control

Setup and decisions

Allocation runs from largest to smallest starting at 10.20.0.0, using the smallest prefix p with 2^(32-p) - 2 at least the hosts needed.

Subnet	Hosts needed	Prefix	Network	Usable range	Broadcast	Usable hosts
app	1,000	/22	10.20.0.0	10.20.0.1 to 10.20.3.254	10.20.3.255	1,022
public	250	/24	10.20.4.0	10.20.4.1 to 10.20.4.254	10.20.4.255	254
data	100	/25	10.20.5.0	10.20.5.1 to 10.20.5.126	10.20.5.127	126
management	20	/27	10.20.5.128	10.20.5.129 to 10.20.5.158	10.20.5.159	30

Working: app needs 1,000 hosts: /23 gives 510 (too few), /22 gives 1,022. public needs 250: /25 gives 126, /24 gives 254. data needs 100: /26 gives 62, /25 gives 126. management needs 20: /28 gives 14, /27 gives 30. Each block starts on a multiple of its own size, so no padding is needed.

NACL for the `public` subnet (`network/nacl.json`):

Rule	Direction	Action	Protocol / ports	Source or destination
100	in	deny	all	198.51.100.0/24
110	in	allow	TCP 80	0.0.0.0/0
120	in	allow	TCP 443	0.0.0.0/0
130	in	allow	TCP 22	10.20.5.128/27 (management)
140	in	allow	TCP 1024-65535	10.20.0.0/22 (app, replies)
100	out	allow	TCP 8000	10.20.0.0/22 (app)
110	out	allow	TCP 1024-65535	0.0.0.0/0 (replies to clients)

Measured results

`python tools/nacl_sim.py network/nacl.json network/packets_sample.jsonl --check`:

```
 S01  inbound  tcp port 443   192.0.2.55      -> ALLOW (rule 120)  OK
 S02  inbound  tcp port 22    192.0.2.55      -> DENY  (no match (default deny))  OK
 S03  outbound tcp port 8000  10.20.1.1       -> ALLOW (rule 100)  OK
 S04  inbound  tcp port 443   198.51.100.9    -> DENY  (rule 100)  OK
 S05  outbound tcp port 60000 192.0.2.55      -> ALLOW (rule 110)  OK
 S06  inbound  tcp port 5432  10.20.5.20      -> DENY  (no match (default deny))  OK

Correct: 6 / 6 = 100.0%
```

Why replies need rules, and why order matters

A NACL is stateless: it does not remember that a request was allowed, so every packet is judged on its own. A client's request to TCP 443 is covered by an inbound rule, but the reply is a different packet on an ephemeral port (1024 to 65535) and needs its own outbound rule. The same applies to replies from the `app` subnet to the load balancer. Rules are evaluated from the lowest number upward and the first match wins. The deny for 198.51.100.0/24 must have a lower number than the allows for ports 80 and 443; with a higher number, a packet from that range to port 443 would match the allow first and the deny would never apply.

6. Task 6: Reproducibility, testing and CI/CD

Setup and decisions

Everything is in the repository. `make up` builds and deploys the whole system; `make down` deletes the cluster. The workflow runs, in order: lint, unit tests, model evaluation (downloads the model from the Release, runs `score.py` on `validation.jsonl`), quality gate (`score.py --min-validity 95`), plus a gitleaks secret scan.

Measured results

Item	Evidence
Unit tests	11 tests: 11 passed, 14 warnings in 0.42 s
`AI_USE.md`	present in repository

7. Task 7: Performance, cost and review

Measured results

Standard load test (4 users, 2 minutes, through the Ingress, 2 replicas). Files: `results/loadtest_stats.csv`, `results/loadtest_headers.json`.

Metric	Value	Brief's check
Requests / failures	509 / 5	
Error rate	0.98%	at most 1%: met
p50 latency	910 ms	
p95 latency	1,300 ms	at most 5,000 ms: met
Throughput	4.28 requests/s	
Cold start (median, Task 4)	5.0 s	

`kubectl -n app top pods` during a repeat of the standard load test (same settings, results written elsewhere so the figures above are unchanged), about one minute in:

```
NAME                      CPU(cores)   MEMORY(bytes)
parser-cdcdcb959-4ntnh   996m         395Mi
parser-cdcdcb959-mmtng   997m         449Mi
```

Cost. Rate card (hypothetical): 3.50 INR per vCPU-hour, 0.45 INR per GiB-hour; each replica is billed for its limits.

1. Hourly cost = replicas x (CPU limit x 3.50 + memory limit x 0.45) = 2 x (1 x 3.50 + 1 x 0.45) = 2 x 3.95 = 7.90 INR per hour.
2. Requests per hour = 4.276 x 3600 = 15,395.4.
3. Cost per 1,000 requests = 7.90 / 15,395.4 x 1000 = 0.5131 INR.

(`python scripts/cost.py --replicas 2 --cpu 1 --mem-gib 1 --stats results/loadtest_stats.csv --slo 99`: hourly cost 7.9000 INR; 15395.4 requests per hour; 0.5131 INR per 1,000 requests; error budget 5.09 requests, used 5 (98.2%))

SLO. Objective: 99% of requests succeed. Window: one load-test run (509 requests). Error budget = 509 x 1% = 5.09 requests. Used: 5 failures = 98.2% of the budget.

Review table

Component or concern	Evidence (measured number)	Limitation found	Improvement proposed
Data and storage	kept 1,312 of 3,000; rule (a) removed 1,551	Rule (a) discards every reply that is not strictly one JSON object (1,551 of 3,000 teacher replies); storage runs on the single cluster node	Make the teacher stop after the JSON so fewer replies are discarded; replicate the storage
Model quality	student full-frame 41%, intent 70%, validity 99% (validation, n = 100)	Intent accuracy caps full-frame; validity 99% means about 1 request in 100 returns HTTP 500	More or cleaner training data; constrained decoding to guarantee valid JSON
Latency and throughput	p50 910 ms, p95 1,300 ms, 4.28 requests/s (4 users, 2 replicas)	Each request takes about 0.9 s on CPU, so throughput is capped by CPU	Shorten generation, quantise the model, or allow more replicas
Resource utilisation	Pods at 996m and 997m CPU (limit 1 core), 395Mi and 449Mi memory (limit 1Gi)	CPU-bound: both Pods run at their CPU limit while memory is under half the limit	Raise the CPU limit or the HPA maximum; lower the memory limit towards 512Mi
Reliability and recovery	rolling update 2.17% failures, recovery 1.01%, cold start median 5.0 s	Single node, so a Pod restart is not a machine failure; the rolling update missed the 0.5% target	Use a multi-node cluster; add a preStop delay so terminating Pods leave the ingress before they exit (14 of the 29 rollout failures were HTTP 502)
Security and cost	0.5131 INR per 1,000 requests	The estimate depends on a hypothetical rate card; both replicas are billed at full limits even when idle	Scale to the HPA minimum when idle; right-size the limits

Assumptions and interpretation

The cost is an estimate from a hypothetical rate card; no real charge applies. Throughput comes from one laptop that also runs the load generator and the cluster, so a real cloud would differ.

8. AI use

Claude (Anthropic) was used to scaffold and review the repository (teacher worker, cleaner, training script, service, manifests, CI workflow, and a first draft of the subnet table and NACL) and to help debug environment problems; `AI_USE.md` lists the details. Claude also drafted the structure of this report and entered the measured values from my result files into it. I checked the work by running it: `clean.py` twice with identical sha256 values, `nacl_sim.py` (6 of 6), the unit tests (11 passed) and all the Kubernetes experiments. Where the AI's expectations were wrong, the measurements showed it: it expected the teacher to out-score the student once its trailing notes were cut, but the teacher reached 40% full-frame against the student's 41%; and it expected cleaning to discard about two thirds of the labels, but 1,551 of 3,000 (51.7%) were removed.